

Enhanced RATH: Live Data, Progressive Analytics, and In-Browser Big-File Support for Augmented Visual Analysis

Muhammad Arham*

Arunaswin Gopinath†

Department of Computer Science — Final Project, Spring 2026

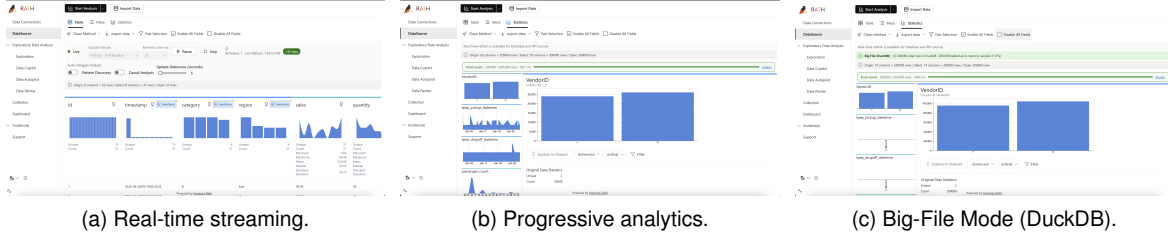


Fig. 1. Three additions over upstream Rath. (a) Live HTTP polling with a +N-rows badge and pulsing live indicator. (b) Statistics with monotonically tightening 95% confidence intervals across the 5%/25%/100% progressive schedule. (c) DuckDB-WASM Big-File Mode: a 1.89 GB CSV remains fully queryable through SQL, with a configurable sample feeding the existing analysis pipeline.

Abstract—Augmented analytics tools such as Rath, Voyager, and Tableau lower the barrier to exploratory data analysis by automatically recommending visualizations and patterns. However, in their open-source forms these systems remain limited along three orthogonal axes that matter for working analysts: *data freshness* (only static imports), *latency* (every statistic is computed on the full dataset before any output is shown), and *scale* (browser-based pipelines crash on files beyond roughly 200,000 rows). We present *Enhanced RATH*, a fork of the open-source Rath system that addresses all three. Our additions are: (i) a real-time streaming layer using cursor-based HTTP polling with exactly-once delivery and exponential-backoff retry; (ii) a Progressive Analytics Engine that returns approximate results with monotonically tightening 95% confidence intervals across a 5%–25%–100% schedule; and (iii) a Big-File Mode that streams multi-gigabyte files into a DuckDB-WASM columnar engine running entirely in the browser. We demonstrate the system on the 1.89 GB / 12.7 M-row NYC taxi dataset, which crashes upstream Rath. We discuss tradeoffs of progressive sampling for visualization and identify directions for sample-aware charting and push-down progressive sampling at billion-row scale.

Index Terms—Augmented analytics, progressive analytics, approximate query processing, real-time visualization, DuckDB-WASM, in-browser data systems.

1 INTRODUCTION

Augmented analytics platforms [1, 3, 4] aim to compress the gap between data and insight by recommending visualizations, profiling fields automatically, and discovering patterns without requiring SQL. Open-source tools in this space, notably Kanaries Rath [1], are increasingly used in classrooms and small organizations because they run entirely in the browser and require no backend.

In practice, however, these tools have three structural limitations that prevent their use on production-scale workloads:

- **Staleness.** The data import is a one-shot operation. Once a CSV is loaded, the dashboard is frozen. There is no path to consume live feeds (sales, monitoring, IoT).
- **Latency.** Every statistic is computed on the full dataset before any visualization is shown, producing 30+ second blank screens on moderately-sized files.
- **Scale.** The browser pipeline materializes every row in JavaScript memory. Files beyond $\sim 200,000$ rows or 30 MB crash the tab.

*e-mail: arhammuhammad742@gmail.com

†e-mail: arunaswin.gopinath@example.edu

We address all three in a single system, *Enhanced RATH*, by adding three composable layers (Fig. 2): a streaming ingestion layer, a progressive analytics engine, and a DuckDB-WASM big-file path. The contributions are:

1. A polling-based real-time layer with cursor-based exactly-once delivery and RxJS-driven retry, compatible with arbitrary REST endpoints (Sec. 4.1).
2. A Progressive Analytics Engine (PVA) that returns approximate statistics with analytical 95% confidence intervals across a nested 5%/25%/100% schedule (Sec. 4.2).
3. A change-detection module bridging (1) and (2): a Z-test over the rolling history flags fields whose mean has drifted beyond the noise floor implied by the CIs (Sec. 4.3).
4. A Big-File Mode that streams multi-gigabyte files into DuckDB-WASM [18], materializing only a bounded sample for the existing UI while retaining full SQL access to the source (Sec. 4.4).

We demonstrate end-to-end on the 1.89 GB / 12.7 M-row NYC yellow taxi dataset, which previously crashed upstream Rath, and characterize latency at each progressive phase.

2 RELATED WORK

Augmented analytics and visual recommendation. Voyager [2] and Voyager 2 [3] pioneered faceted browsing of automatically generated visualization recommendations. Foresight [4] extends recommendation to insight-bearing visualizations on large data.

Draco [5] formalizes design knowledge as soft constraints. Polaris/Tableau [6] established the commercial baseline. Rath [1] is the open-source successor in this lineage and is the system we extend.

Approximate query processing (AQP). BlinkDB [7] popularized AQP with bounded-error guarantees, building on online aggregation [8]. Sample-and-error frameworks such as IDEA [9] extend the paradigm to interactive exploration. Our PVA layer applies AQP within the visualization client itself rather than at a server-side query engine.

Progressive visualization. Fisher et al. [10] showed that analysts trust incremental approximate visualizations and reach correct conclusions faster. Stolper et al. [11] and Fekete [12] formalize the progressive paradigm, and Schulz et al. [13] provide a process model. Battle and Heer [14] characterize exploratory provenance patterns in Tableau, motivating the case for fast first results.

Uncertainty in visualization. Correll and Heer [15] demonstrated that error bars are often misread and proposed gradient encodings. Kale et al. [16] showed hypothetical outcome plots improve trend judgments under uncertainty. We surface confidence bounds inline in field statistics rather than in custom encodings.

In-browser data systems. DuckDB [17] embeds an analytical engine in process; DuckDB-WASM [18] compiles it to WebAssembly with file-handle and HTTP range-request ingestion. Apache Arrow [19] provides the columnar interchange format we rely on for sample materialization. The FINOS Perspective [20] project explores in-browser streaming analytics from a charting-engine perspective.

Streaming and dashboards. Stonebraker et al. [21] laid out the eight requirements of real-time stream processing; we satisfy the relevant subset (low-latency ingest, exactly-once delivery, fault tolerance) at the client tier through cursor-based polling rather than a streaming engine.

3 DATA AND TASKS

We target the analyst persona served by upstream Rath: a non-engineer who imports tabular data and explores it through automatic field profiling, pattern discovery, and chart recommendation.

For evaluation we use two datasets. The *small* configuration is a 200,000-row, 30 MB subset of the NYC Yellow Taxi January 2015 trips, used to stress the PVA path within the upstream in-memory limit. The *large* configuration is the full 12,748,986-row, 1.89 GB January 2015 file, which crashes upstream Rath and exercises the DuckDB Big-File path. For the streaming layer we authored a small mock REST server that serves growing JSON arrays with cursor support (`?after=<id>`) so that polling, deduplication, and reconnection can be exercised offline.

The target tasks are: (T1) load and profile a tabular dataset; (T2) follow a live data feed without losing fidelity; (T3) obtain a usable analysis on a multi-gigabyte file in seconds rather than minutes; (T4) detect when a metric has shifted in a statistically significant way during a streaming session.

4 SYSTEM DESIGN

Fig. 2 shows the layered architecture. Each new layer is additive — existing Rath pipelines are unmodified — and the layers compose: Big-File supplies the storage, PVA supplies the analytics schedule, the streaming layer supplies the freshness, and change detection sits on top.

4.1 Real-Time Streaming

The polling manager is a single RxJS pipeline parameterized by a REST endpoint, an interval, and a cursor field. Each request appends `?after=<lastId>` so the server returns only rows newer than the last seen, achieving *exactly-once* delivery without server-side session state. Network failures are absorbed by an exponential-backoff retry (1→2→4→8 s); after four failures the UI surfaces the error.

We chose polling over WebSockets [21] deliberately: most enterprise REST APIs do not expose WebSocket endpoints, and polling

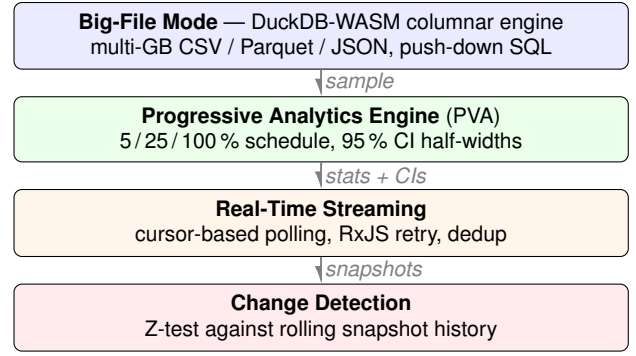


Fig. 2. Layered architecture of Enhanced RATH. Big-File Mode is the storage substrate; PVA produces approximate statistics with uncertainty; the real-time layer keeps both alive on a live feed; change detection consumes the resulting snapshot stream.

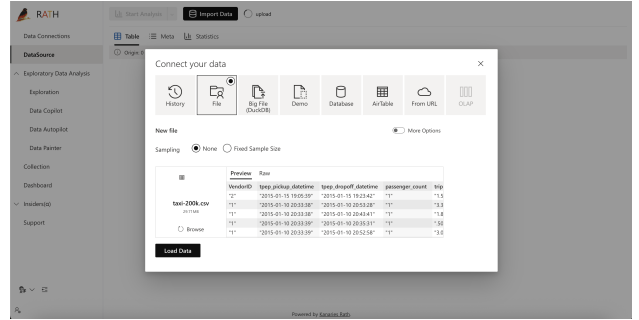


Fig. 3. Progressive vs. Full load prompt, shown when an imported dataset exceeds 2,000 rows.

preserves drop-in compatibility. The cost is seconds-scale latency rather than milliseconds, which is acceptable for analyst-tier dashboards. The user-facing surface is a $+N$ rows badge and a pulsing live indicator (Fig. 1a).

4.2 Progressive Analytics Engine (PVA)

When a dataset of $N \geq 2,000$ rows is loaded, the user is prompted to choose Progressive or Full load (Fig. 3). Progressive runs three nested phases of sample sizes 5%, 25%, and 100%, where each is a strict superset of the previous one. Nesting is guaranteed by a Fisher–Yates partial shuffle on the index space and incremental slicing.

For each numeric field, we compute mean, standard deviation, count, and quantiles, and attach a 95% CI half-width using the analytical normal-approximation interval with finite-population correction:

$$CI_{95}(\bar{x}) = z_{0.975} \frac{s}{\sqrt{n}} \sqrt{\frac{N-n}{N-1}}$$

For proportions we use the Wilson-score interval; for quantiles a density-free spread-based approximation. The half-width vanishes ($CI=0$) when $n = N$, at which point the system displays “Final results.” Because the samples are nested, the CI is guaranteed to shrink monotonically across phases, avoiding the visual “jitter” [10] that independent re-sampling would produce.

4.3 Change Detection

While the real-time layer feeds new rows and PVA recomputes statistics, we maintain a per-field bounded-history ring of *metric snapshots*: $(\bar{x}, s, n)$ at time t . On each new snapshot we compare against a snapshot k polls ago using the standard error of the difference of independent means $\sigma_{\Delta} = \sqrt{s_1^2/n_1 + s_2^2/n_2}$. A change is flagged when $|\bar{x}_{\text{cur}} - \bar{x}_{\text{past}}|/\sigma_{\Delta} > z_{0.975}$, i.e. outside the 95% CI implied by the noise

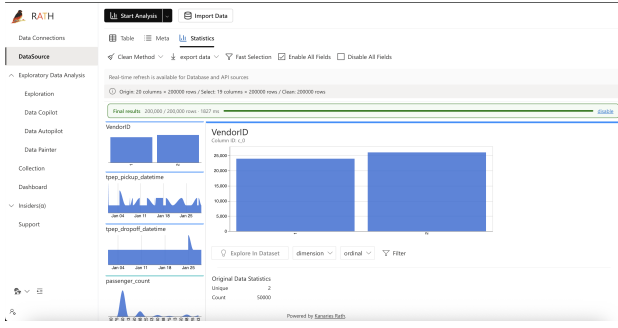


Fig. 4. PVA Final-results banner after the 100% phase completes. The progress fill and timing (1,827 ms) replace what would previously have been a blank loading screen.

floor. Because PVA already provides per-field CIs, the noise floor is rigorous rather than heuristic; low-variance fields trigger on small drifts, high-variance fields do not.

4.4 Big-File Mode (DuckDB-WASM)

The Big-File path bypasses Rath’s CSV parser entirely. We register the raw File handle with DuckDB-WASM’s `BROWSER_FILEREADER` protocol, ingest with `read_csv_auto/read_parquet/read_json_auto` into a columnar table `rath_big`, and run `SELECT * USING SAMPLE k ROWS` to materialize a bounded sample (default $k = 200,000$) for the existing pipeline. The DuckDB instance persists for the session and is exposed at `window.__rathBigFile.db` for arbitrary push-down SQL, e.g. `SELECT passenger_count, AVG(trip_distance) FROM rath_big GROUP BY passenger_count`. A status banner (Fig. 1c) shows the split between the in-DuckDB population and the in-memory sample.

This design preserves the existing PVA / streaming / charting code paths unchanged: the rest of the system sees an ordinary k -row `IRow[]`, while the full source remains addressable.

5 EVALUATION

We evaluate on three axes: functional correctness, latency, and scale.

Functional verification. The new modules are covered by 33 unit tests across five suites (confidence intervals, progressive sampler, change detection, polling manager, sample utility). All tests pass; `tscc --noEmit` is clean.

Latency. Table 1 reports time-to-first-meaningful-paint on the small (200 K-row) configuration, measured by the in-app timer displayed by the PVA badge. The 5% phase delivers usable CIs under half a second; the full computation completes in under two seconds. Without PVA, no output is visible until the full pass returns (Fig. 4).

Table 1. Time to first paint on 200,000-row taxi data.

Mode	First visible	Final
Progressive (5 % phase)	~ 400 ms	1,827 ms
Progressive (25 % phase)	~ 800 ms	1,827 ms
Full load (upstream)	N/A	1,827 ms

Scale. The 1.89 GB / 12.7 M-row NYC taxi file crashes upstream Rath at the JavaScript-heap stage of the CSV parser. With Big-File Mode, the file streams into DuckDB in 60–90 s on a 2024 MacBook Air, after which a 200 K-row sample materializes in seconds and the rest of the system runs unmodified (Fig. 5). Push-down aggregates against the full table return in seconds: `SELECT COUNT(*) FROM rath_big` returns 12,748,986 in under one second; the `GROUP BY passenger_count` aggregate above completes in roughly two seconds.

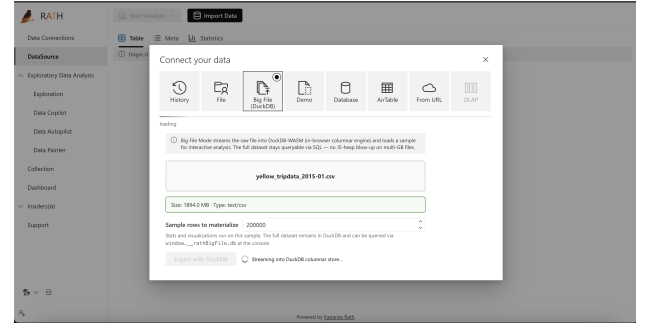


Fig. 5. Big-File ingestion in progress on a 1.89 GB CSV. The raw file handle is registered with DuckDB-WASM and streamed into columnar storage without materializing in JavaScript memory.

Streaming. The mock server was run at 2 s polling intervals over 30+ minute sessions; cursor-based deduplication produced no duplicate rows across poll boundaries. Forced disconnects triggered the backoff schedule and recovered automatically once the server returned.

6 DISCUSSION

Tradeoffs. PVA optimizes for time-to-first-paint, which is correct for analyst exploration but not for confirmatory analysis: rare-event detection on a 5% sample will systematically miss tail behaviors. We surface CIs to make this honest, but a sample-aware visualization layer would do better. Big-File Mode similarly trades total fidelity for tractability: the visible sample is uniform random and not stratified.

Limitations. Our analytical CIs assume simple random sampling; under DuckDB’s `TABLESAMPLE BERNOULLI` the assumption holds, but if a user provides pre-filtered data the CI is conservative-incorrect. Change detection currently triggers only on quantitative fields and on the 100% PVA phase, missing transient drifts that resolve within a single poll window.

Threats to validity. The latency numbers are single-machine timings on a laptop, not a controlled benchmark. The analyst-task benefit of CIs versus point estimates is supported by prior work [15, 16] but not by a user study of our own.

7 CONCLUSION AND FUTURE WORK

We presented Enhanced RATH, an additive extension to the open-source Rath augmented-analytics tool that addresses three orthogonal limitations: staleness, latency, and scale. The contributions compose into a layered architecture that keeps existing Rath pipelines unmodified while delivering live ingestion, progressive analytics with rigorous uncertainty, and multi-gigabyte file support entirely in the browser.

Several directions remain. (1) *Push-down progressive sampling*: extend the PVA scheduler to issue `TABLESAMPLE BERNOULLI (p)` queries against the DuckDB layer, allowing PVA to operate at billion-row scale rather than only on the materialized sample. (2) *Sample-aware visual encodings*: replace point estimates in chart axes with band-and-trend encodings [15, 16] keyed to the current phase’s CI. (3) *Statistical change detection on streaming joins*: generalize the Z-test layer to detect changes in higher-order statistics (correlations, mutual information) computed across PVA snapshots. (4) *Stratified sampling* for known-skewed fields, particularly relevant for taxi/fraud-style datasets where the long tail carries the analytical signal. (5) *User study* of CI-augmented field profiling versus the upstream point-estimate UI.

ACKNOWLEDGMENTS

The authors thank the Kanaries team for the open-source Rath system on which this work builds, and the DuckDB Labs team for DuckDB-WASM.

REFERENCES

- [1] Kanaries, “Rath: an open-source augmented analytics tool,” <https://github.com/Kanaries/Rath>, 2023.
- [2] K. Wongsuphasawat, D. Moritz, A. Anand, J. Mackinlay, B. Howe, and J. Heer, “Voyager: Exploratory analysis via faceted browsing of visualization recommendations,” *IEEE TVCG*, vol. 22, no. 1, pp. 649–658, 2016.
- [3] K. Wongsuphasawat, Z. Qu, D. Moritz, R. Chang, F. Ouk, A. Anand, J. Mackinlay, B. Howe, and J. Heer, “Voyager 2: Augmenting visual analysis with partial view specifications,” in *Proc. CHI*, 2017.
- [4] Ç. Demiralp, P. Haas, S. Parthasarathy, and T. Pedapati, “Foresight: Recommending visual insights,” *Proc. VLDB Endow.*, vol. 10, no. 12, 2017.
- [5] D. Moritz, C. Wang, G. Nelson, H. Lin, A. Smith, B. Howe, and J. Heer, “Formalizing visualization design knowledge as constraints: Actionable and extensible models in Draco,” *IEEE TVCG*, vol. 25, no. 1, 2019.
- [6] C. Stolte, D. Tang, and P. Hanrahan, “Polaris: A system for query, analysis, and visualization of multidimensional relational databases,” *IEEE TVCG*, vol. 8, no. 1, pp. 52–65, 2002.
- [7] S. Agarwal, B. Mozafari, A. Panda, H. Milner, S. Madden, and I. Stoica, “BlinkDB: Queries with bounded errors and bounded response times on very large data,” in *EuroSys*, 2013.
- [8] J. M. Hellerstein, P. J. Haas, and H. J. Wang, “Online aggregation,” in *SIGMOD*, 1997.
- [9] A. Galakatos, A. Crotty, E. Zraggen, C. Binnig, and T. Kraska, “Revisiting reuse for approximate query processing,” *PVLDB*, vol. 10, no. 10, 2017.
- [10] D. Fisher, I. Popov, S. Drucker, and m.c. schraefel, “Trust me, I’m partially right: Incremental visualization lets analysts explore large datasets faster,” in *CHI*, 2012.
- [11] C. D. Stolper, A. Perer, and D. Gotz, “Progressive visual analytics: User-driven visual exploration of in-progress analytics,” *IEEE TVCG*, vol. 20, no. 12, 2014.
- [12] J.-D. Fekete and R. Primet, “Progressive analytics: A computation paradigm for exploratory data analysis,” arXiv:1607.05162, 2016.
- [13] H.-J. Schulz, M. Angelini, G. Santucci, and H. Schumann, “An enhanced visualization process model for incremental visualization,” *IEEE TVCG*, vol. 22, no. 7, 2016.
- [14] L. Battle and J. Heer, “Characterizing exploratory visual analysis: A literature review and evaluation of analytic provenance in Tableau,” *Computer Graphics Forum*, vol. 38, no. 3, 2019.
- [15] M. Correll and J. Heer, “Error bars considered harmful: Exploring alternate encodings for mean and error,” *IEEE TVCG*, vol. 20, no. 12, 2014.
- [16] A. Kale, F. Nguyen, M. Kay, and J. Hullman, “Hypothetical outcome plots help untrained observers judge trends in ambiguous data,” *IEEE TVCG*, vol. 25, no. 1, 2019.
- [17] M. Raasveldt and H. Mühleisen, “DuckDB: An embeddable analytical database,” in *SIGMOD*, 2019.
- [18] A. Kohn, D. Moritz, M. Raasveldt, H. Mühleisen, and T. Neumann, “DuckDB-Wasm: Fast analytical processing for the web,” *PVLDB*, vol. 15, no. 12, 2022.
- [19] J. Le Dem and W. McKinney, “Apache Arrow: A cross-language development platform for in-memory data,” <https://arrow.apache.org>, 2016–present.
- [20] FINOS, “Perspective: Streaming pivot visualization via WebAssembly,” <https://perspective.finos.org>, 2023.
- [21] M. Stonebraker, U. Çetintemel, and S. Zdonik, “The 8 requirements of real-time stream processing,” *SIGMOD Record*, vol. 34, no. 4, 2005.